

Java Backend Architecture

Interview Series

Chapter 12.8

Multi-Tenancy & Locale Per User

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

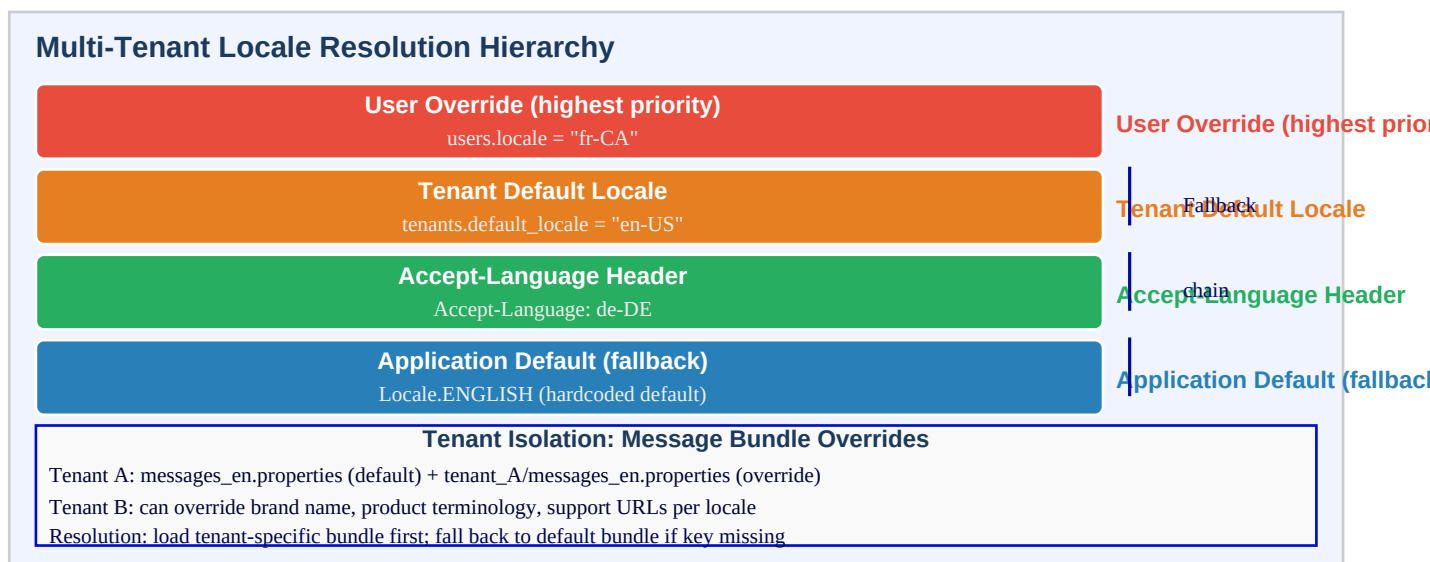
Chapter 12.8 – Multi-Tenancy & Locale Per User

Introduction

In a multi-tenant SaaS application, locale is not a single global setting — it operates at multiple levels simultaneously. A tenant (an organisation using the platform) may configure a default locale for all its users, overriding the application default. Each individual user may further override the tenant default with their personal preference, stored in the users table as a locale column (e.g., "fr-CA" for Canadian French). The locale resolution chain evaluates these layers in order of priority: user override, tenant default, Accept-Language header, and finally the application default. Spring's `LocaleContextHolder` stores the resolved locale in a `ThreadLocal`, making it available throughout the request processing pipeline without passing it as a parameter to every method.

The primary challenge in multi-tenant localisation is ensuring that async processing — scheduled jobs, `@Async` service methods, and reactive WebFlux pipelines — correctly carries the locale from the original request context. `ThreadLocal` does not propagate across thread boundaries, so dedicated propagation mechanisms are required: a `TaskDecorator` on the thread pool executor for `@Async`, or `Reactor Context` for WebFlux Mono and Flux chains. Including the locale in JWT claims provides a convenient mechanism to propagate user preferences across service boundaries in a microservices architecture. Message bundle caching eliminates repeated file-system reads, but multi-tenant applications must also support tenant-specific message overrides — where a tenant can customize brand names or terminology in their locale bundles.

Multi-Tenant Locale Resolution Hierarchy



LocaleContextHolder: ThreadLocal vs Async/Reactive

LocaleContextHolder: ThreadLocal in Sync vs Async/Reactive

Synchronous (Servlet)

Request → DispatcherServlet sets locale
in LocaleContextHolder (ThreadLocal)
Works out of the box; cleared after request

Async / Reactive

ThreadLocal NOT propagated by default!
Fix: TaskDecorator for @Async thread pools
Fix: Reactor Context for WebFlux Mono/Flux

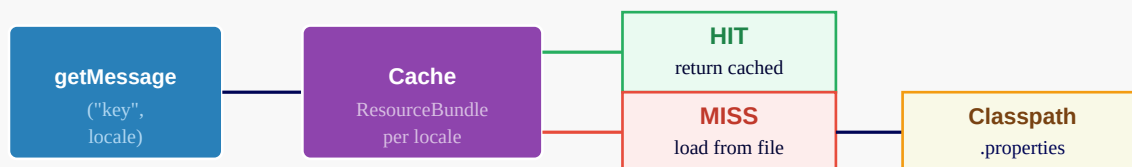
Solutions for Async Locale Propagation

```
@Async: ThreadPoolTaskExecutor.setTaskDecorator(r -> { Locale lc = LocaleContextHolder.getLocale();
    return () -> { LocaleContextHolder.setLocale(lc); try { r.run(); }
        finally { LocaleContextHolder.resetLocaleContext(); } }; })
WebFlux: Mono.deferContextual(ctx -> { Locale l = ctx.get("locale"); ... })
Entry point: .contextWrite(Context.of("locale", locale))
```

JWT: include locale claim; read at entry point filter; propagate to Reactor Context or TaskDecorator

Message Bundle Caching Strategy

Message Bundle Caching & Performance



Spring MessageSource Cache Configuration

```
spring.messages.cache-duration=-1 # -1 = cache forever (production default)
spring.messages.cache-duration=0 # 0 = reload every request (dev mode, hot-reload)
spring.messages.cache-duration=3600 # 1 hour TTL (hot-reload in production)
Tenant override: load bundles from DB at startup; cache per tenantId+locale key in ConcurrentHashMap
```

ReloadableResourceBundleMessageSource: watches filesystem; useful for external translation files updated at runtime

Key Definitions

Multi-tenancy	Architecture where a single application instance serves multiple independent organisations (tenants), each with isolated data and configuration
Tenant locale	A default locale configured at the tenant (organisation) level, applied to all users of that tenant who have not set a personal locale preference
User locale preference	A locale stored per user (e.g., users.locale = "fr-CA") that overrides the tenant default; highest priority in the locale resolution chain
LocaleContextHolder	Spring utility class storing the current Locale in a ThreadLocal; set by DispatcherServlet per request; accessible anywhere in the call stack
LocaleChangeInterceptor or	Spring MVC HandlerInterceptor that reads a locale query parameter (?lang=ar) and calls LocaleResolver.setLocale() to update the stored locale
CookieLocaleResolver	Spring LocaleResolver storing the locale in a browser cookie; persists across sessions; allows locale changes via LocaleChangeInterceptor

SessionLocaleResolver	Spring LocaleResolver storing the locale as an HTTP session attribute; cleared when the session expires or is invalidated
TaskDecorator	Spring interface for wrapping Runnable tasks submitted to async executors; used to propagate ThreadLocal context (locale, security) to new threads
Reactor Context	Immutable key-value store attached to a Reactor pipeline (Mono/Flux); the reactive alternative to ThreadLocal for carrying context across async operations
Tenant message override	Per-tenant customization of message bundle values (e.g., brand name, product terminology) loaded in addition to the default locale bundles
JWT locale claim	Custom claim in a JWT token (e.g., "locale": "fr-CA") allowing microservices and async workers to read the user's locale without a database lookup
ReloadableResourceBundleMessageSource	Spring MessageSource that watches the filesystem for .properties file changes; enables hot-reload of translations without restarting the app
cache-duration	spring.messages.cache-duration property: -1 = cache forever (production); 0 = no cache (dev/hot-reload); positive value = TTL in seconds
Locale fallback chain	Resolution order when a message key is missing: fr-CA → fr → default; Spring MessageSource applies this automatically
Thread pool isolation	In multi-tenant async contexts: each @Async thread pool may need to be tenant-aware; the TaskDecorator propagates tenantId alongside locale

Locale Resolver Comparison

Resolver / Source	Storage	Override via	Auth required	Multi-tenant use
AcceptHeaderLocaleResolver	HTTP header	Header change	No	API clients manage their own
CookieLocaleResolver	Browser cookie	?lang= param	No	Good for anonymous users
SessionLocaleResolver	HTTP session	?lang= param	No	Requires session management
DB user preference	users table	Profile update	Yes	Best for SaaS, multi-tenant
JWT claim	JWT token	Re-issue JWT	Yes	Microservices, stateless
Tenant config	tenants table	Admin setting	Yes	Tenant-level default

Code Examples

1. Storing and resolving user locale preference from database

```
// User entity with locale preference
@Entity @Table(name = "users") public class User { @Id private Long id; private String email; @Column(name = "locale", length = 20) private String locale = "en-US"; // e.g. "fr-CA", "ar-SA", "de-DE"
public Locale getPreferredLocale() { if (locale == null || locale.isBlank()) return Locale.ENGLISH; return Locale.forLanguageTag(locale); // BCP 47 tag parsing } }

// Tenant entity with default locale
@Entity @Table(name = "tenants") public class Tenant { @Id private Long id; private String name; @Column(name = "default_locale", length = 20) private String defaultLocale = "en-US";
public Locale getDefaultLocale() { return Locale.forLanguageTag(defaultLocale); } }

// Custom locale resolver: user preference > tenant default > Accept-Language
@Component public class MultiTenantLocaleResolver implements LocaleResolver { @Autowired UserRepository userRepo;
```

```
@Autowired TenantRepository tenantRepo; @Autowired AcceptHeaderLocaleResolver headerResolver;
@Override public Locale resolveLocale(HttpServletRequest req) { // 1. Authenticated user
    preference Authentication auth = SecurityContextHolder.getContext().getAuthentication(); if
    (auth != null && auth.getPrincipal() instanceof UserPrincipal principal) { User user =
    userRepo.findById(principal.getUserId()).orElse(null); if (user != null && user.getLocale() !=
    null) { return user.getPreferredLocale(); } // 2. Tenant default Tenant tenant =
    tenantRepo.findByUser(user); if (tenant != null) return tenant.getDefaultLocale(); } // 3.
    Accept-Language header fallback return headerResolver.resolveLocale(req); } @Override public
    void setLocale(HttpServletRequest req, HttpServletResponse res, Locale locale) { // Save new
    locale to user profile Authentication auth =
    SecurityContextHolder.getContext().getAuthentication(); if (auth != null &&
    auth.getPrincipal() instanceof UserPrincipal principal) {
    userRepo.updateLocale(principal.getUserId(), locale.toLanguageTag()); } } }
```

2. LocaleContextHolder usage and ThreadLocal lifecycle

```
import org.springframework.context.i18n.LocaleContextHolder; // Reading locale anywhere in the
call stack (same thread as request) @Service public class NotificationService { @Autowired
    MessageSource messages; public String buildWelcomeMessage(String username) { Locale locale =
    LocaleContextHolder.getLocale(); // ThreadLocal – no parameter needed String template =
    messages.getMessage("notification.welcome", null, locale); return template.replace("{name}",
    username); } // Explicitly setting locale (e.g., in batch jobs) public void
    processForLocale(Locale locale, Runnable task) { LocaleContextHolder.setLocale(locale); try {
    task.run(); } finally { LocaleContextHolder.resetLocaleContext(); // CRITICAL: clean up after
    task } } } // Spring Security filter: set locale after authentication @Component public class
    LocalePropagationFilter extends OncePerRequestFilter { @Autowired UserRepository userRepo;
    @Override protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
    FilterChain chain) throws IOException, ServletException { Authentication auth =
    SecurityContextHolder.getContext().getAuthentication(); if (auth != null &&
    auth.isAuthenticated()) { Long userId = ((UserPrincipal) auth.getPrincipal()).getUserId();
    userRepo.findById(userId).ifPresent(user -> {
    LocaleContextHolder.setLocale(user.getPreferredLocale()); }); } } try { chain.doFilter(req,
    res); } finally { LocaleContextHolder.resetLocaleContext(); } } }
```

3. Locale propagation in @Async with TaskDecorator

```
import org.springframework.core.task.TaskDecorator; // TaskDecorator: propagates locale (and
security context) to @Async threads @Component public class LocaleAndSecurityTaskDecorator
implements TaskDecorator { @Override public Runnable decorate(Runnable runnable) { // Capture
    from the calling thread Locale locale = LocaleContextHolder.getLocale(); SecurityContext
    securityContext = SecurityContextHolder.getContext(); return () -> { // Set on the worker
    thread LocaleContextHolder.setLocale(locale);
    SecurityContextHolder.setContext(securityContext); try { runnable.run(); } finally { // Clean
    up after execution LocaleContextHolder.resetLocaleContext();
    SecurityContextHolder.clearContext(); } } } } // Register TaskDecorator on async executor
@Configuration @EnableAsync public class AsyncConfig { @Bean(name = "asyncExecutor") public
    Executor asyncExecutor(LocaleAndSecurityTaskDecorator decorator) { ThreadPoolTaskExecutor
    executor = new ThreadPoolTaskExecutor(); executor.setCorePoolSize(5);
    executor.setMaxPoolSize(20); executor.setQueueCapacity(500);
    executor.setThreadNamePrefix("async-"); executor.setTaskDecorator(decorator); // ← propagates
    locale + security executor.initialize(); return executor; } } // Service: locale now available
in @Async method @Service public class EmailService { @Async("asyncExecutor") public void
    sendWelcomeEmail(Long userId) { Locale locale = LocaleContextHolder.getLocale(); // available
    thanks to TaskDecorator String subject = messages.getMessage("email.welcome.subject", null,
    locale); // send email with locale-appropriate content... } }
```

4. Locale in JWT claims for microservices

```
import io.jsonwebtoken.*; // Include locale in JWT when issuing token
@Service public class JwtService { private static final String LOCALE_CLAIM = "locale"; public String
generateToken(User user) { return Jwts.builder() .setSubject(user.getId().toString())
.claim("email", user.getEmail()) .claim("tenantId", user.getTenantId().toString())
.claim(LOCALE_CLAIM, user.getLocale()) // e.g. "fr-CA" .claim("roles", user.getRoles())
.setIssuedAt(new Date()) .setExpiration(Date.from(Instant.now().plus(Duration.ofHours(24))))
.signWith(secretKey, SignatureAlgorithm.HS256) .compact(); } public Locale
extractLocale(String token) { Claims claims = Jwts.parserBuilder() .setSigningKey(secretKey)
.build() .parseClaimsJws(token) .getBody(); String localeTag = claims.get(LOCALE_CLAIM,
String.class); return localeTag != null ? Locale.forLanguageTag(localeTag) : Locale.ENGLISH; //
fallback } } // JWT authentication filter: set locale from JWT claim public class
JwtAuthenticationFilter extends OncePerRequestFilter { @Override protected void
doFilterInternal(HttpServletRequest req, ...) { String token = extractToken(req); if (token !=
null) { Locale locale = jwtService.extractLocale(token);
LocaleContextHolder.setLocale(locale); // set for this request thread } chain.doFilter(req,
res); } } // In microservice B (downstream): receives JWT, extracts locale // No DB call needed
- locale is embedded in the token // Consistent locale across service boundaries in a
microservices architecture
```

5. Locale propagation in WebFlux / Project Reactor

```
import reactor.core.publisher.Mono; import reactor.util.context.Context; // Reactive locale
propagation using Reactor Context @Component public class ReactiveLocaleContextFilter
implements WebFilter { @Autowired ReactiveLocaleResolver localeResolver; @Override public
Mono<Void> filter(ServerWebExchange exchange, WebFilterChain chain) { return
localeResolver.resolveLocale(exchange) .flatMap(locale -> chain.filter(exchange)
.contextWrite(Context.of("locale", locale)) // attach to Reactor Context ); } } // Reading
locale in a reactive service @Service public class ReactiveProductService { @Autowired
MessageSource messages; public Mono<ProductDto> getProduct(Long id) { return
Mono.deferContextual(ctx -> { Locale locale = ctx.getOrDefault("locale", Locale.ENGLISH); //
read from context return productRepository.findById(id) .map(p -> toDto(p, locale)); }); }
private ProductDto toDto(Product p, Locale locale) { String desc =
messages.getMessage("product.desc." + p.getCategory(), null, locale); return new
ProductDto(p.getId(), p.getName(), desc); } } // Custom ReactiveLocaleResolver @Component
public class ReactiveLocaleResolver { @Autowired ReactiveUserRepository userRepo; public
Mono<Locale> resolveLocale(ServerWebExchange exchange) { // Try JWT claim first return
Mono.justOrEmpty(exchange.getRequest().getHeaders().getFirst("Authorization"))
.map(this::extractLocaleFromJwt) .switchIfEmpty(Mono.fromSupplier(() -> // Fall back to
Accept-Language exchange.getRequest().getHeaders().getAcceptLanguage() .stream().findFirst()
.map(r -> Locale.forLanguageTag(r.getRange())) .orElse(Locale.ENGLISH) )); } }
```

6. Tenant-specific message overrides and bundle caching

```
// Multi-tenant MessageSource: tenant overrides + global fallback @Configuration public class
MultiTenantMessageSourceConfig { // Tenant overrides loaded at startup and cached per tenantId
+ locale @Bean public MessageSource messageSource(TenantMessageRepository tenantMsgRepo) {
return new MultiTenantMessageSource(tenantMsgRepo); } } @Component public class
MultiTenantMessageSource implements MessageSource { private final TenantMessageRepository
tenantMsgRepo; // Cache: tenantId -> locale -> key -> message private final Map<String,
Map<Locale, Map<String, String>>> cache = new ConcurrentHashMap<>(); private final
ResourceBundleMessageSource globalBundle; public
MultiTenantMessageSource(TenantMessageRepository repo) { this.tenantMsgRepo = repo;
this.globalBundle = new ResourceBundleMessageSource(); globalBundle.setBasename("messages");
globalBundle.setDefaultEncoding("UTF-8"); globalBundle.setCacheSeconds(-1); // cache forever }
```

```
@Override public String getMessage(String code, Object[] args, String defaultMsg, Locale locale) {
    String tenantId = TenantContextHolder.getTenantId(); // ThreadLocal // Check tenant
    override cache first String override = cache .computeIfAbsent(tenantId, t ->
    loadTenantMessages(t)) .getOrDefault(locale, Map.of()) .get(code); if (override != null) {
    return args != null ? MessageFormat.format(override, args) : override; } // Fall back to global
    bundle return globalBundle.getMessage(code, args, defaultMsg, locale); } private Map<Locale,
    Map<String,String>> loadTenantMessages(String tenantId) { // Load from DB: SELECT locale,
    message_key, message_value // FROM tenant_messages WHERE tenant_id = ? return
    tenantMsgRepo.findByTenantId(tenantId).stream() .collect(Collectors.groupingBy( m ->
    Locale.forLanguageTag(m.getLocale()), Collectors.toMap(TenantMessage::getKey,
    TenantMessage::getValue) )); } // Cache eviction when tenant admin updates messages public void
    evictTenantCache(String tenantId) { cache.remove(tenantId); } @Override public String
    getMessage(MessageSourceResolvable resolvable, Locale locale) { return
    getMessage(resolvable.getCodes()[0], resolvable.getArguments(),
    resolvable.getDefaultMessage(), locale); } @Override public String getMessage(String code,
    Object[] args, Locale locale) { return getMessage(code, args, code, locale); } }
```

Interview Questions & Answers

Q1. How does locale resolution work in a multi-tenant application?

Multi-tenant locale resolution follows a priority chain: (1) User personal preference — stored in users.locale column (e.g., 'fr-CA'); highest priority if the user is authenticated and has set a preference. (2) Tenant default locale — stored in tenants.default_locale; applies to users of that tenant who haven't set a personal preference. (3) Accept-Language HTTP header — the browser/client's preferred language for anonymous or unauthenticated access. (4) Application default — hardcoded fallback (usually Locale.ENGLISH). Implementation: a custom LocaleResolver that queries the user's locale from the database post-authentication, otherwise falls back down the chain. The resolved locale is stored in LocaleContextHolder for the request duration.

Q2. What is LocaleContextHolder and how does its ThreadLocal work?

LocaleContextHolder is a Spring utility that stores the current Locale in a ThreadLocal. ThreadLocal stores a separate value per thread — each HTTP request thread in a servlet container has its own locale. Lifecycle: DispatcherServlet calls LocaleContextHolder.setLocaleContext() at the start of each request using the configured LocaleResolver. Throughout the request (controller, service, repository) LocaleContextHolder.getLocale() retrieves it without parameter passing. At the end of the request, LocaleContextHolder.resetLocaleContext() clears it. The ThreadLocal must be reset after use to prevent memory leaks in thread pools where threads are reused across requests.

Q3. Why doesn't ThreadLocal work in @Async methods and how do you fix it?

When Spring's @Async proxy invokes a method, it submits a Runnable to a thread pool executor. The worker thread is different from the request thread — ThreadLocal values (including LocaleContextHolder and SecurityContextHolder) are NOT copied to the new thread. Fix: implement TaskDecorator and register it on the ThreadPoolTaskExecutor. The decorator captures the ThreadLocal values from the calling thread before the task runs, sets them on the worker thread, and clears them after the task completes (in a finally block). Spring Security provides DelegatingSecurityContextRunnable for the security context; you must write the locale propagation yourself or use a combined decorator that handles both.

Q4. How do you propagate locale in a Spring WebFlux reactive application?

WebFlux uses Project Reactor's Mono and Flux, which may switch threads during execution. ThreadLocal is unreliable across thread switches. Solution: Reactor Context — an immutable key-value map attached to the

reactive pipeline. At the entry point (a `WebFilter`), resolve the locale and attach it to the context: `chain.filter(exchange).contextWrite(Context.of('locale', resolvedLocale))`. In downstream services, read it: `Mono.deferContextual(ctx -> { Locale l = ctx.getOrDefault('locale', Locale.ENGLISH); ... })`. Key difference from `ThreadLocal`: `Reactor Context` is immutable and propagates 'upstream' (from subscriber to publisher), unlike `ThreadLocal` which propagates along the thread. Spring Security's `ReactiveSecurityContextHolder` uses the same mechanism.

Q5. How should the user's locale preference be stored in the database?

Store as a `VARCHAR(20)` column (`users.locale`) using BCP 47 language tags: 'en-US', 'fr-CA', 'ar-SA', 'zh-CN'. BCP 47 is the standard used by Java's `Locale.toLanguageTag()` and `Locale.forLanguageTag()`. Avoid storing as separate language and country columns — a combined tag is easier to work with. Validation: when saving, parse with `Locale.forLanguageTag()` and verify the result is not the 'und' (undetermined) locale. Optionally restrict to supported locales: reject locales not in the application's supported list. Index: if querying users by locale (e.g., 'find all French users for a campaign'), add an index on the locale column. Default: store `NULL` or 'en-US' for users who haven't set a preference.

Q6. How do you include locale in a JWT token and why is it useful?

Add a custom 'locale' claim when issuing the JWT: `Jwts.builder().claim('locale', user.getLocale().toLanguageTag())`. Benefits: (1) Stateless — any microservice that receives the JWT can determine the user's locale without a DB call. (2) Cross-service consistency — all services in a request chain use the same locale. (3) Async workers — batch jobs that process user data can extract the locale from a stored JWT or user snapshot. Downsides: (1) Locale change requires a new JWT (or the change is delayed until token expiry). (2) The locale claim adds bytes to every request header. For short-lived JWTs (15 min) this is acceptable; for long-lived tokens, also store locale in the database as the authoritative source.

Q7. How do you implement tenant-specific message bundle overrides?

Architecture: (1) Global bundle: `messages.properties` and `messages_fr.properties` in classpath, covering all generic messages. (2) Tenant overrides: a database table `tenant_messages(tenant_id, locale, message_key, message_value)` or per-tenant properties files in an external directory. (3) Custom `MessageSource`: check tenant overrides first (using the `tenantId` from `TenantContextHolder`), then fall back to the global bundle. (4) Cache: a `ConcurrentHashMap` avoids repeated DB queries; cache is loaded at startup and evicted when a tenant admin updates messages. Use case: Tenant A has rebranded 'Item' as 'Asset'; Tenant B uses standard terminology. Each sees their own translation for the same message key.

Q8. What is the message bundle caching configuration in Spring Boot?

`spring.messages.cache-duration` controls how long Spring caches loaded `ResourceBundle` instances. Values: -1 (default in production): cache forever — bundles are loaded once at first use and never reloaded; 0: no caching — reload from filesystem every request; useful in development with hot-reload. Positive integer (seconds): reload after the TTL expires; enables hot-reload of external translation files in production. `ReloadableResourceBundleMessageSource` supports this and can load from file: or classpath: locations, watching for changes. For multi-tenant with database overrides: manage your own `ConcurrentHashMap` cache with explicit eviction via an admin API (`POST /admin/cache/locale/evict?tenantId=X`).

Q9. How does Spring's locale fallback chain work for message resolution?

When `getMessage('welcome', null, Locale.forLanguageTag('fr-CA'))` is called: (1) Spring looks for `messages_fr_CA.properties` — if found and key exists, return it. (2) Fall back to `messages_fr.properties` — the language-only bundle. (3) Fall back to `messages.properties` — the default bundle. (4) If still not found: throw `NoSuchMessageException` (if configured) or return the default message parameter. This hierarchy is defined by `ResourceBundleMessageSource` following Java's `ResourceBundle` lookup convention. Implication: you must have a fallback bundle (`messages.properties`) with all keys; locale-specific bundles only need to

contain keys that differ from the default. This also means a missing translation silently falls back to English rather than showing an error.

Q10. What are the risks of not resetting `LocaleContextHolder` after use?

`LocaleContextHolder` uses a `ThreadLocal`. In thread pools (used by every servlet container and `@Async` executor), threads are reused across requests. If the locale is set but not cleaned up after a request, the next request served by that thread will inherit the previous request's locale — a thread pollution bug. This causes intermittent locale errors: a French user might see English, or an Arabic user sees French. It is also a security-adjacent concern if locale contains tenant information. Fix: always wrap locale-setting code in try-finally: `LocaleContextHolder.setLocale(locale); try { ... } finally { LocaleContextHolder.resetLocaleContext(); }`. `DispatcherServlet` handles this for normal request processing; you must handle it in custom filters and `TaskDecorators`.

Q11. How do you test locale-per-user behavior in Spring Boot tests?

1. MockMvc with Accept-Language header: `mockMvc.perform(get('/').header('Accept-Language', 'fr')).andExpect(content().string(containsString('Bonjour')))`; 2. MockMvc with a fake authenticated user: use `@WithMockUser` and a custom `UserDetails` that carries the locale. 3. Direct `LocaleContextHolder` setup in unit tests: `LocaleContextHolder.setLocale(Locale.FRENCH); try { ... } finally { LocaleContextHolder.resetLocaleContext(); }`. 4. Test the locale resolution chain: mock `UserRepository.findById` to return a user with `locale='de-DE'`; assert `MessageSource` uses German. 5. Test fallback: remove `messages_de.properties` keys; assert fallback to `messages.properties`. 6. Parameterized test across all supported locales to ensure no missing keys.

Q12. How should a locale change be applied for a logged-in user?

UI: provide a language selector (flag icons or locale names). On selection: (1) Send a PATCH `/api/users/me/locale {locale: 'fr-CA'}` request to the backend. (2) Backend validates locale (must be in supported list), saves to `users.locale`, and returns a new JWT with the updated locale claim. (3) Frontend stores the new token, reloads the page with the new locale. Alternative for session-based apps: POST `/locale?lang=fr` triggers `LocaleChangeInterceptor` → `setLocale()` → saves to session and database. For immediate effect without full reload: JavaScript Intl API can format numbers and dates client-side in the new locale while the backend processes the preference asynchronously. Do not require the user to log out and back in.

Q13. What is `TenantContextHolder` and how does it interact with locale?

Similar to `LocaleContextHolder`, `TenantContextHolder` is typically a custom `ThreadLocal`-based holder storing the current tenant ID for multi-tenant applications. It works in parallel with `LocaleContextHolder`: the request-processing filter sets both the `tenantId` (from JWT or subdomain) and the locale (from the resolution chain). The `MultiTenantMessageSource` reads both: `tenantId` to look up the tenant override bundle, and locale to select the language. Both `ThreadLocals` must be propagated together in `TaskDecorators` and must be reset together in finally blocks. Some implementations combine them into a single `TenantLocaleContext` object stored in one `ThreadLocal` to simplify propagation.

Q14. How do you handle locale in scheduled batch jobs (`@Scheduled`)?

`@Scheduled` methods run on a task scheduler thread that has no request context — `LocaleContextHolder` is empty. Options: (1) For jobs that process all users, iterate over users grouped by locale, set `LocaleContextHolder.setLocale(userLocale)` for each batch, and reset after. (2) For jobs that generate content for a specific locale (e.g., monthly report), set the target locale explicitly at the start of the job method. (3) For jobs that send user-specific notifications: query each user's locale and apply it per user within the loop. Always reset in finally. (4) For multi-tenant jobs: also set the `tenantId` in `TenantContextHolder`. Document which locale is used for system-generated audit logs (typically UTC-based timestamps with a system locale,

not user locale).

Q15. How do you handle locale for email notifications generated asynchronously?

Email notifications are typically generated in an `@Async` context or by a scheduled job, not during a user request. Patterns: (1) Capture locale at event publication time: `ApplicationEventPublisher.publishEvent(new WelcomeEvent(userId, locale))`; the event payload carries the locale so the async handler has it. (2) Store locale in the notification table: `notifications(user_id, type, locale, ...)` and read it when processing. (3) For background workers (Kafka consumers, job processors): include locale in the message/job payload explicitly — do not rely on `ThreadLocal`. Template engines: Thymeleaf `TemplateEngine.process(template, context)` accepts a `Locale` in the context: `context.setLocale(userLocale)`.

Q16. How do you test locale propagation in `@Async` methods?

1. Configure a synchronous executor for tests: `@Bean @Primary public Executor asyncExecutor() { return Runnable::run; }` — this makes `@Async` methods run synchronously in the same thread, so `ThreadLocal` propagation is automatic. Test the business logic; test the propagation mechanism separately. 2. Test `TaskDecorator` explicitly: `LocaleContextHolder.setLocale(Locale.FRENCH); executor.execute() -> assertThat(LocaleContextHolder.getLocale()).isEqualTo(Locale.FRENCH)`; 3. For reactive code: use `StepVerifier` with `contextWrite`: `StepVerifier.create(service.process()).contextWrite(Context.of('locale', Locale.FRENCH)).expectNextMatches(result -> result.getLanguage().equals('fr')).verifyComplete()`; 4. Always assert `ThreadLocal` is clean after async execution.

Q17. What Spring properties configure the `MessageSource` for multi-tenant i18n?

`spring.messages.basename`: comma-separated list of bundle base names; e.g., `'messages,tenant-messages'` to load from multiple locations. `spring.messages.encoding`: UTF-8 (required for non-Latin translations). `spring.messages.fallback-to-system-locale`: false (prevent using the server's OS locale as fallback; always use `Locale.ENGLISH` from the configured default). `spring.messages.use-code-as-default-message`: false (throw `NoSuchMessageException` for missing keys in production to catch translation gaps early). `spring.messages.always-use-message-format`: false (disable `MessageFormat` parsing for messages without placeholders — performance optimization). For external files: use `ReloadableResourceBundleMessageSource` with `setBasenames('file:/app/config/messages', 'classpath:messages')`.

Q18. How does locale affect validation error messages in Spring?

Spring Validation (`javax.validation` / `jakarta.validation` with Hibernate Validator) uses a `MessageInterpolator` to resolve constraint message templates. By default, Hibernate Validator uses the JVM default locale for messages. In Spring Boot with i18n: configure `LocalValidatorFactoryBean` with the Spring `MessageSource` so validation messages (`@NotNull`, `@Size`) are resolved from your `messages.properties` bundles. The message key format: `@NotBlank message='{user.name.required}'`; in `messages_fr.properties`: `user.name.required=Le nom est obligatoire`. The current locale is taken from `LocaleContextHolder` at the time of validation — important: validation triggered from `@Async` methods needs locale propagation.

Q19. How do you handle user locale preference changes without requiring re-login?

The challenge: if the locale is embedded in the JWT, changing the locale requires a new JWT. Approaches: (1) Short TTL JWTs (15 min) with refresh tokens — locale change triggers immediate re-issue of access token with new locale claim. (2) Store locale in session, not JWT — session is mutable; `SessionLocaleResolver.setLocale()` updates immediately. (3) Hybrid: locale not in JWT; resolved from DB on each request using `userId` from JWT; slightly more DB load but locale changes are instant. (4) Client-side: after saving the locale preference, the client applies the new locale immediately for UI (Intl API) and sends `?lang=fr-CA` on subsequent requests. Best UX: apply the change without page reload using JavaScript, confirm server persistence asynchronously.

Q20. What performance considerations apply to per-request locale resolution from a database?

Loading a user's locale from the database on every request adds a DB query per request. Mitigations: (1) Include locale in the JWT (as described) — eliminates the DB call. (2) Cache in the HTTP session (SessionLocaleResolver): resolved once per session, not per request. (3) L1 cache: Hibernate second-level cache or Spring Cache (@Cacheable on the user profile read) — locale changes rarely. (4) Redis/Memcached: cache the user's locale string by userId with a TTL matching typical session duration. (5) Read replica: locale reads are trivially consistent and can go to a read replica, not the primary DB. (6) Materialized locale in the principal: load locale into UserDetails at authentication time (Spring Security) so it's available for the entire session without additional DB calls.